ChatGPT

CNF-DampSine：阻尼振荡函数重建与物理硬约束实验设计报告

Executive Summary

执行优先级：先做一个最小可行实验（MVP），确认 CNF 的硬约束链条完全正确，再扩大到四方法对照和鲁棒性扫描。 CNF 的核心不是“把 PDE/ODE residual 放进 loss”，而是将线性微分算子作用到一组可学习基函数上，构造约束矩阵 A_f，每次前向或训练迭代通过可微线性求解得到系数 β，从而在指定配置点上显式满足约束。论文明确支持恒等、任意阶微分以及多个线性算子联合约束，并在训练过程中反复求解 β。 [1]

MVP 建议固定为： 单条 FunDiff `damp_sine`，无噪声；$K = 33$，其中内部 ODE 点 $M = 31$，再加入左右两个 Dirichlet 硬边界；先跑 `CNF-Gaussian`，随后跑 `CNF-SIREN`，最后加入 Fourier collocation 与 SIREN-PINN。FunDiff 原始生成器定义 $x \in [0, 1]$，默认使用均匀传感器点，参数范围为 $A \in [0.5, 1]$、$\gamma \in [2, 4]$、$\omega \in [6\pi, 8\pi]$、shift $b \in [-0.5, 0.5]$；原函数为 $Ae^{-\gamma x}\sin(\omega x) + b$。 [2]

推荐工程路线：单独建立 PyTorch 环境实现 CNF-DampSine，不在第一阶段移植到 FunDiff/JAX。 原始 CNF 官方实现本身是 PyTorch/PyTorch Lightning，并使用 `torch.linalg.solve`、autograd 高阶导数和 `functorch`；官方仓库锁定了 `torch==1.13.0` 与 `pytorch-lightning==1.8.1`。FunDiff 则使用 JAX/Flax，两者强行共用环境会增加与研究问题无关的兼容性成本。 [3]

研究假设应分开验证： CNF-Gaussian 测试论文推荐的标准核基函数；CNF-SIREN 测试周期型神经基函数；Fourier collocation 是固定谱基硬约束基线；SIREN-PINN 是可学习正弦网络的软约束基线。论文明确指出 Gaussian kernel 在标准规模、无重复 anchor 的场景是推荐方案，而重复位置施加多个算子时普通 kernel 会产生重复基函数，因此需要 hypernetwork kernel 或重新设计约束位置。 [4]

超参数与实验配置

下表区分“**首选值**”与“**扫描值**”。没有直接由原论文指定、而是针对 DampSine 提出的值，均标注为“本实验建议”，不能写成 CNF 论文原始超参数。

维度	MVP / 首选配置	正式扫描配置	说明与依据
数据函数	$f(x) = Ae^{-\gamma x}\sin(\omega x) + b$	不变	FunDiff 官方数据生成式。 [2]
A	U(0.5,1.0)	不变	官方范围。 [2]
γ	U(2,4)	不变	官方范围。 [2]
ω	U$(6\pi, 8\pi)$	OOD 可增加 $5\pi-6\pi$, $8\pi-9\pi$	官方训练分布为 $6\pi-8\pi$。 [2]
b	U(-0.5,0.5)	不变	FunDiff 源码中名为 `shift`。 [2]

维度	MVP / 首选配置	正式扫描配置	说明与依据
每条函数原始采样点	128	128	FunDiff autoencoder config 默认 `num_sensors=128`；生成器本身默认参数为 100。为与正式 FunDiff 配置一致，本实验建议统一使用 128。 [5]
dense test grid	2048	4096 可选	本实验建议；用于防止只在配置点上评估
Train / Val / Test 函数数	800 / 100 / 100	1600 / 200 / 200	本实验建议；参数空间独立随机采样
MVP 函数数	1	—	固定一组参数，先验证数值正确性
随机种子	42	42, 123, 2026, 3407, 7777	FunDiff 默认 seed=42；正式统计建议 5 seeds。 [6]
保存真实参数	**是**	是	必须保存 A, γ, ω, b，否则无法构造样本特定 ODE
观测点数 N_{obs}	32	16 / 32 / 64	本实验建议
噪声	0%	0 / 1 / 5%	建议定义 $\epsilon \sim \mathcal{N}(0, (\eta\,\mathrm{std}(f))^2)$
CNF 总基函数数 K	**33**	17 / 33 / 65	对应 $M+2$，保证第一阶段方阵
内部 ODE 点 M	**31**	15 / 31 / 63	与两个边界组成 $K = M + 2$
ODE 点位置	Chebyshev interior	uniform / random / Chebyshev	MVP 首选 Chebyshev，正式做分布消融
CNF-Gaussian anchor	每个约束一 anchor	同约束位置	符合 kernel-basis 的 anchor 设计。 [4]
Gaussian σ	0.05	0.01 / 0.025 / 0.05 / 0.1 / 0.2	**本实验建议**；官方代码提供 `init_sigma` 参数，但论文并未为本任务指定值。 [7]
Gaussian encoder	PositionalEncoder, `n_freqs=3`	identity / n_freqs=1,3,5	官方 `kernel` 初始化使用 `PositionalEncoder(..., n_freqs=3)`，也支持 identity encoder。 [7]
Gaussian basis 参数	**学习**	fixed / learned 两组	CNF 的关键是学习基函数 inductive bias；β 不作为 optimizer 参数，而由线性系统求解。 [4]
CNF-SIREN basis	hypernet_siren	sepbasis_siren 作为消融	官方 CNF 代码直接包含 `hypernet_siren` 和 `sepbasis_siren`。 [7]

维度	MVP / 首选配置	正式扫描配置	说明与依据
CNF-SIREN hidden	21	21 / 32 / 64	官方 `hypernet_siren` 使用 width 21、1 hidden layer。 [7]
CNF-SIREN layers	1 hidden	1 / 2 / 3	官方默认 1；本实验可扫描。 [7]
CNF-SIREN ω_0	30	10 / 20 / 30 / 40	官方 CNF SIREN 实现默认 `first_omega_0=30`，`hidden_omega_0=30`；SIREN 官方实现采用相同核心设计。 [8]
SIREN-PINN width	64	32 / 64 / 128	本实验建议
SIREN-PINN hidden layers	3	2 / 3 / 4	本实验建议
SIREN-PINN ω_0	30	10 / 20 / 30 / 40	SIREN 官方代码/论文将第一层频率因子 30 作为标准设置，并采用专门初始化。 [9]
Fourier K	33 列	17 / 33 / 65	为公平起见矩阵列数与 CNF 相同
Fourier 频率	$0, \pi, 2\pi, \ldots, 40\pi$ 候选库	max 16π / 24π / 40π	**本实验建议**。目标真实频率在 $6\pi-8\pi$，40π 是超完备压力测试，不是先验答案
Fourier 步长	π	$\pi/2, \pi, 2\pi$	本实验建议；使用 sin/cos 成对列
optimizer	Adam	AdamW 作消融	SIREN 官方工作使用专门初始化配合梯度优化；CNF 官方 PyTorch 模型采用 Lightning/optimizer 框架。 [10]
CNF LR	1×10^{-3}	$3e{-}4, 1e{-}3, 3e{-}3$	本实验建议
SIREN-PINN LR	1×10^{-4}	$1e{-}4, 3e{-}4, 1e{-}3$	本实验建议
最大训练 steps	CNF 5k；PINN 10k	CNF 10k；PINN 20k	本实验建议
early stop	patience 500	patience 1000	监控 val Relative-L2，同时要求约束误差不过阈值
cond regularizer	$10^{-6} \log \kappa(A)$ 起步	$0, 10^{-8}, 10^{-6}, 10^{-4}$	CNF 论文明确推荐在许多任务中正则化条件数，但具体权重应为任务消融。 [1]

一个重要设计决定是：**正式数据文件要比 FunDiff 原生成器多保存四列参数。** 原始 `generate_dataset()` 只返回 `x,y`，虽然内部随机采了 $A, \gamma, \omega, \text{shift}$，但没有返回这些参数；我们的数据包装器应返回 `x, y_clean, y_noisy, params=[A,gamma,omega,b]`。 [2]

方法、ODE 与硬边界设计

DampSine 写成

$$f(x) = Ae^{-\gamma x}\sin(\omega x) + b, \qquad q(x) = f(x) - b.$$

直接求导得到

$$q'' + 2\gamma q' + (\gamma^2 + \omega^2)q = 0.$$

因此定义样本特定线性算子

$$\mathcal{L}_{\gamma,\omega} = \frac{d^2}{dx^2} + 2\gamma\frac{d}{dx} + (\gamma^2 + \omega^2)I.$$

它完全落在 CNF 论文允许的线性算子框架内：论文允许 identity、任意阶微分以及由多个微分项组合的线性算子，并允许多个不同算子同时约束一个 neural field。 4

CNF-Gaussian

模型写成

$$q_\theta(x) = \sum_{j=1}^{K} \beta_j \Psi_j(x;\theta),$$

Gaussian kernel basis 采用论文定义

$$\Psi_j(x) = \kappa(\phi_\theta(a_j), \phi_\theta(x)),$$

其中 a_j 是 anchor，ϕ_θ 为可学习 encoder。论文提出 kernel basis 的目的之一就是提升基函数线性独立性；论文明确推荐在“标准规模且无重复 anchor 多重约束”时使用普通 Gaussian kernel basis。 4

MVP 中取 $M = 31$ 个内部点

$$x_m \in (0,1),$$

再加入两个 Dirichlet 边界：

$$q(0) = 0, \qquad q(1) = Ae^{-\gamma}\sin(\omega).$$

于是 $K = M + 2 = 33$，构造

$$A = \begin{bmatrix} \mathcal{L}\Psi_1(x_1) & \cdots & \mathcal{L}\Psi_K(x_1) \\ \vdots & & \vdots \\ \mathcal{L}\Psi_1(x_M) & \cdots & \mathcal{L}\Psi_K(x_M) \\ \Psi_1(0) & \cdots & \Psi_K(0) \\ \Psi_1(1) & \cdots & \Psi_K(1) \end{bmatrix},$$

$$g = [0,\dots,0,q(0),q(1)]^\top, \qquad A\beta = g.$$

这意味着 **ODE 与 boundary 都是硬约束，而不是 loss**。CNF 论文的矩阵构造正是将不同 operator/constraint-point 逐行堆叠后求解 basis weights；官方实现中的 `compute_weights()` 直接调用 `torch.linalg.solve(X, constraint_value)`。[11]

齐次 ODE 如果没有额外非零信息，会允许 $q \equiv 0$。这里的右边界通常提供非零值，因此排除了零解。若某个随机样本恰好使 $q(1)$ 极接近零，则 MVP 应自动改用 **初值型硬条件**

$$q(0) = 0, \qquad q'(0) = A\omega,$$

因为解析导数给出 $q'(0) = A\omega$。但这会在同一个 $x = 0$ 上同时约束 value 与 derivative；论文明确警告普通 kernel basis 在重复 anchor 上施加多个约束会造成重复基函数和病态系统，因此这个版本应切换到 **hypernetwork kernel / hypernet-SIREN basis**，而不是普通 Gaussian kernel。[4]

CNF-SIREN

这里不是“一个 SIREN 直接输出 $q(x)$”，而是 **SIREN 产生 neural basis，CNF 仍通过 $A\beta = g$ 硬求系数**。官方 CNF 仓库已经提供 `hypernet_siren`：main network 是 width 21、1 hidden layer 的 SIREN，hypernetwork 根据 basis id 产生参数；也提供 `sepbasis_siren` 和更大的 SIREN 变种。[7]

SIREN 的基本单元为

$$h = \sin(\omega_0(Wx + b)).$$

官方 CNF 实现以及 SIREN 官方实现都使用专门初始化；CNF 源码默认 $\omega_0 = 30$，第一层权重和后续层权重采用不同范围，目的是保持 activation 统计并允许有效传播高频梯度。[8]

Fourier collocation

用固定谱基替换神经基函数：

$$q(x) = c_0 + \sum_{n=1}^{P} \left[a_n \sin(\nu_n x) + b_n \cos(\nu_n x)\right].$$

同样对每个基函数解析计算 $\mathcal{L}\phi_j(x_m)$，构造**完全相同的** $A\beta = g$。因此它和 CNF 的差别主要是：Fourier 的 inductive bias 固定，CNF 的 basis shape 可学习。CNF 论文正是把自己的方法解释为“collocation with learnable inductive bias”，而传统 collocation 依赖固定 basis。[4]

为了避免偷偷把真实 ω 告诉 Fourier，主实验**不允许**把每个样本的真实 ω 直接作为单独 Fourier basis。建议固定公共频率库；主配置可取

$$\nu \in \{0, \pi, 2\pi, \ldots, 16\pi\},$$

需要压力测试时扩展至 40π。列数需要与 CNF 的 K 对齐，因此实际选取 sin/cos 列时应固定“总列数”而不是只固定最高频率。

SIREN-PINN

这是唯一的**软约束**主基线：

$$q_\theta(x) = \mathrm{SIREN}_\theta(x)$$

并训练

$$L = \lambda_d L_{\text{data}} + \lambda_r L_{\text{ODE}} + \lambda_b L_{\text{BC}}.$$

主配置建议

$$\lambda_d = 1, \quad \lambda_r = 1, \quad \lambda_b = 100,$$

随后扫描 $\lambda_r \in \{0.1, 1, 10, 100\}$。这里的边界没有“严格满足”的保证；这正是与 CNF 硬约束区分的实验变量。SIREN 原工作强调正弦隐式表示能够有效表示信号本身及其导数，并展示了微分方程/边值问题，因此它是合适的 PINN 型对照。 12

数值求解、训练和实现步骤

CNF 论文描述的训练循环是：每一轮先利用当前 neural basis 构造 A_f，用可微线性求解器得到 β，然后基于其他优化目标对 basis 参数 θ 反向传播。论文使用**带部分主元的 LU 分解和行交换**，并指出只要 A_f 满秩，可微求解就能把 $\partial\beta/\partial\theta$ 纳入计算图。 4 官方当前代码接口则用 `torch.linalg.solve`，并在训练中显式计算 `torch.linalg.cond(X)` 作为 condition-number 项。 13

推荐 solver 策略

主结果必须先使用“原论文语义”的直接求解：`torch.linalg.solve(A,g)`。不要一开始就在主模型中加 ridge，因为 Tikhonov 会把精确约束

$$A\beta = g$$

改成近似约束。论文的核心卖点是硬等式约束，正式结果不能在没有说明的情况下用 ridge 替代。 4

工程上建议设三档条件数告警：

$\kappa_2(A)$	状态	动作
$< 10^6$	正常	LU/`solve`
$10^6 \sim 10^{10}$	警告	保留结果，同时记录；优先调 σ、anchor、basis initialization
$> 10^{10}$ 或 solve fail	失败候选	标记 unstable；尝试 QR/SVD 诊断，不覆盖主结果
$> 10^{12}$	视为严重病态	不允许悄悄继续；报告该 trial 为 numerical failure

这些阈值是**本实验工程规则，不是论文规定值**。论文只明确指出低 condition number 会降低奇异性与解误差界，并建议在很多问题中将其作为正则项。 4

若要做“稳定化 CNF”消融，可另设：

$$\beta_\lambda = (A^\top A + \lambda I)^{-1} A^\top g, \qquad \lambda \in \{10^{-10}, 10^{-8}, 10^{-6}, 10^{-4}\},$$

但必须把这一组标为 **CNF-Tikhonov**，不与严格 CNF 混名。另一个更合适的优先处理顺序是：先重新初始化 basis → 调 σ → 更换 anchor 分布 → condition-number regularization → 最后才 ridge。

可微求解实现要点

官方 CNF 的微分工具通过 `torch.autograd.grad(..., create_graph=True, retain_graph=True)` 构造高阶导数，因此我们构造 Ψ' 和 Ψ'' 时必须保留计算图；否则 basis encoder 无法从训练 loss 接收到梯度。 14

```
def damped_operator(basis_value, x, gamma, omega):
    d1 = torch.autograd.grad(
        basis_value, x,
        grad_outputs=torch.ones_like(basis_value),
        create_graph=True
    )[0]
    d2 = torch.autograd.grad(
        d1, x,
        grad_outputs=torch.ones_like(d1),
        create_graph=True
    )[0]
    return d2 + 2.0 * gamma * d1 + (gamma**2 + omega**2) * basis_value
```

CNF 不应该把 β 放进 Adam 参数列表：

```
A = build_constraint_matrix(...)
beta = torch.linalg.solve(A, g)       # differentiable
pred = basis_matrix(x_query) @ beta
loss = regression_loss(pred, y_obs) + cond_weight * cond_penalty(A)
loss.backward()                       # gradient flows through solve -> A -> basis params
optimizer.step()
```

这种“每个 iteration 重新解 β，只优化 basis 参数”的流程与论文 Algorithm 1 一致。 4

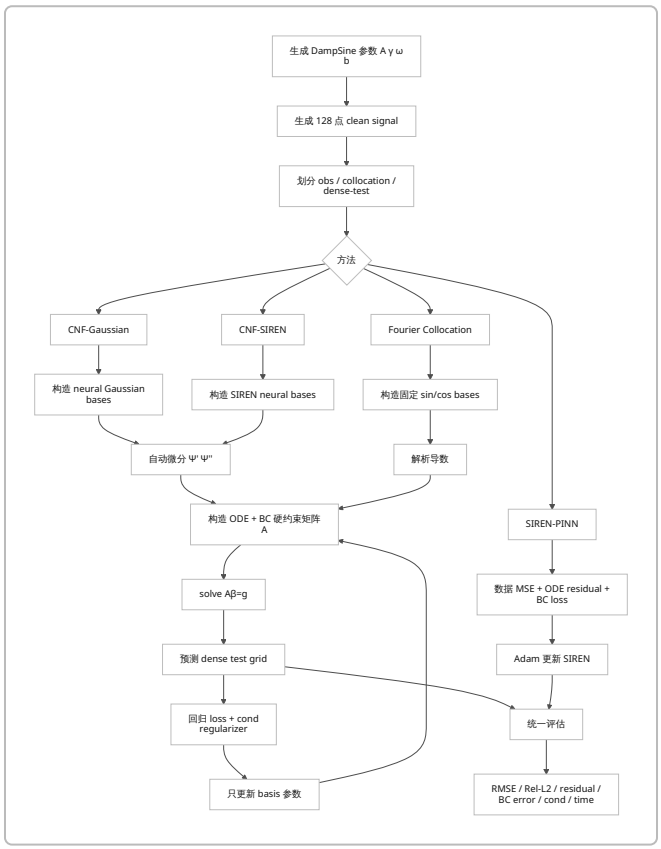
batch 策略

第一阶段逐函数训练。这最接近标准 CNF constrained field：每条函数有自己的 γ, ω, b、边界右端值以及自己的 A_f。MVP 与主论文比较先采用这一策略。

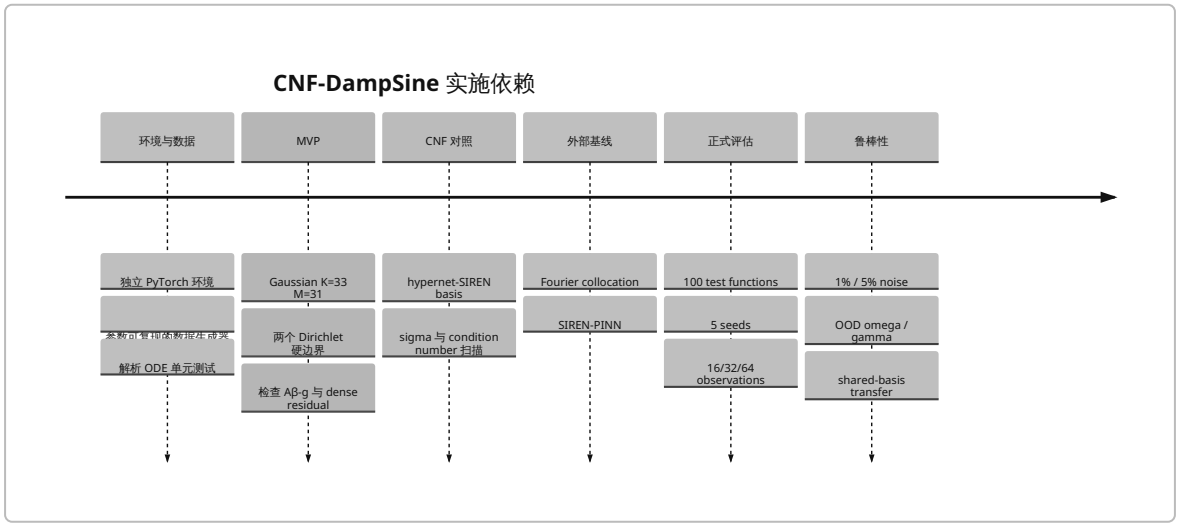
第二阶段再做 shared-basis transfer。 CNF 论文明确提出 basis 可以预训练后迁移至 unseen constraints，只重新求 β，这是非常适合 DampSine 家族实验的扩展：在 800 条 train function 上学习共享 basis inductive bias，在 test functions 上冻结 θ，仅构造新 A 并求 β。 4

SIREN-PINN 则按函数单独训练，避免它获得 CNF 没有的跨样本共享优势。若后续做 amortized multi-function SIREN，应作为独立实验。

流程图



建议时间线



代码组织、接口与训练配置

考虑到官方 CNF 是 PyTorch，而 FunDiff 数据代码虽然位于 JAX 项目中，但生成函数本身只依赖 NumPy，因此建议 **直接复刻数据公式，不 import 整个 FunDiff 包**。这样不会再次触发你此前遇到的 JAX API 兼容问题，同时仍严格保持数据分布。FunDiff 的生成器确实只是 NumPy 采样四个参数并生成阻尼正弦，然后返回数组。 2

建议工程结构：

```
CNF_DampSine/
├── configs/
│   ├── mvp.yaml
│   ├── gaussian.yaml
│   ├── cnf_siren.yaml
│   ├── fourier.yaml
│   └── siren_pinn.yaml
├── data/
│   ├── generator.py
│   ├── dataset.py
│   └── splits/
├── models/
│   ├── bases/
│   │   ├── gaussian_kernel.py
│   │   ├── hypernet_siren.py
│   │   └── fourier_basis.py
│   ├── cnf.py
│   └── siren_pinn.py
├── physics/
│   ├── damped_oscillator.py
│   ├── constraints.py
│   └── collocation.py
├── solvers/
│   ├── differentiable_solve.py
│   └── diagnostics.py
├── training/
│   ├── train_cnf.py
│   ├── train_pinn.py
│   └── early_stop.py
├── evaluation/
│   ├── metrics.py
│   ├── evaluate.py
│   └── plots.py
├── scripts/
│   ├── run_mvp.py
│   ├── run_sweep.py
│   └── aggregate.py
├── tests/
│   ├── test_analytic_ode.py
│   ├── test_derivatives.py
│   └── test_hard_constraints.py
└── results/
    └── <run_id>/
```

关键接口建议保持极小且可替换：

```
def sample_damp_sine(
      n_functions: int,
      n_sensors: int,
      seed: int,
) -> dict:
   """return x, y, A, gamma, omega, b"""

def make_collocation_points(
      M: int,
      kind: str,     # "uniform" | "random" | "chebyshev"
      seed: int,
) -> torch.Tensor:
      ...

class Basis(nn.Module):
      def forward(self, x: torch.Tensor) -> torch.Tensor:
     """return Phi: [N, K]"""

def build_hard_constraint_system(
      basis: Basis,
      x_interior: torch.Tensor,
      gamma: float,
      omega: float,
      boundary_spec: dict,
) -> tuple[torch.Tensor, torch.Tensor]:
   """return A[K,K], g[K]"""

def solve_coefficients(
      A: torch.Tensor,
      g: torch.Tensor,
      method: str = "lu",
) -> torch.Tensor:
      ...

def evaluate_dense(
      model,
      sample: dict,
      n_grid: int = 2048,
) -> dict:
      ...
```

实现时不要重新发明 CNF 的关键设计。官方代码已经提供 `KernelBasis`、`HypernetBasis`、`SeparateBasis`、`hypernet_siren` 初始化、Gaussian kernel encoder、自动微分算子和可微 `torch.linalg.solve`；这些文件应作为实现行为的首要参考。[15]

训练建议如下：

方法	被 optimizer 更新的参数	β	初始 LR	训练步数	早停
CNF-Gaussian	feature encoder、可选 sigma	每步 solve	1e-3	5k	val Rel-L2 500 steps 无改善
CNF-SIREN	hypernet + SIREN basis 参数	每步 solve	3e-4	5k–10k	同上
Fourier	无神经参数	单次 solve	—	0	—
SIREN-PINN	全部 SIREN 权重	无	1e-4	10k–20k	val loss 1000 steps

SIREN 初始化必须沿用其专门初始化；官方 CNF 的 `SineLayer` 第一层使用 $[-1/n, 1/n]$，后续层按 $\sqrt{6/n}/\omega_0$ 缩放，默认 $\omega_0 = 30$。[16]

实验矩阵与优先运行顺序

不要直接做四方法 × 所有参数的笛卡尔积；那会把大量算力浪费在还没有验证正确性的配置上。建议分层推进。

优先级	实验	方法	K	M	N_{obs}	noise	目的
P0	MVP-A	CNF-Gaussian	**33**	**31**	32	0	验证硬 ODE + 硬 BC
P0	MVP-B	CNF-SIREN	**33**	**31**	32	0	验证 neural sinusoidal basis
P0	MVP-C	Fourier	**33**	**31**	32	0	固定谱基直接对照
P0	MVP-D	SIREN-PINN	width 64	31	32	0	软约束基线
P1	Capacity	Gaussian/ SIREN/ Fourier	17,33,65	$K-2$	32	0	容量与 condition
P1	Obs	四方法	33	31	16,32,64	0	稀疏观测
P1	Collocation	四方法	33	15,31,63	32	0	约束密度
P1	Point layout	CNF/ Fourier	33	31	32	0	uniform/random/ Chebyshev
P1	Gaussian sigma	CNF-Gaussian	33	31	32	0	$\sigma = .01, .025, .05, .1, .2$
P1	SIREN omega	CNF-SIREN + PINN	33/ width64	31	32	0	$\omega_0 = 10, 20, 30, 40$

优先级	实验	方法	K	M	N_{obs}	noise	目的
P2	Noise-low	四方法	33	31	32	1%	噪声鲁棒性
P2	Noise-high	四方法	33	31	32	5%	噪声鲁棒性
P2	OOD-frequency	四方法	33	31	32	0	ω 外推
P2	OOD-decay	四方法	33	31	32	0	γ 外推
P3	Transfer	CNF-Gaussian/SIREN	33	31	32	0	shared learned basis，test 时仅 solve β

噪声实验必须区分“哪些数据是硬约束”。**不能把 noisy observations 全部直接作为硬 equality，再宣称模型抗噪。** 如果 noisy value 被写进 $A\beta = g$，CNF 的任务就是精确拟合 noisy constraint；这体现的是 interpolation，不是 denoising。噪声主实验应让 ODE 与解析 boundary 保持 clean hard constraints，而 noisy observations 只进入 regression objective。CNF 论文正是把 hard constraints 与其他 optimization loss 分离：basis weights 负责约束，额外训练参数负责学习约束点之外的 inductive bias。 [4]

正式统计建议每个主配置使用 100 条 test functions × 5 seeds。对于 P2/P3 可先跑 20 条 smoke test，只有没有 numerical failure 后再跑完整 100 条。

评估、可视化与结果表

主评估必须在**独立的 dense grid** 上，而不是只在 M 个 collocation points 上。CNF 只能保证给定离散 constraints 在数值精度下成立；论文的形式化条件也是在离散集合 $\mathcal{S}$ 上构造矩阵方程。 [4] 因此必须同时报告“约束点误差”和“全区间 residual”。

核心指标：

$$\mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_i (\hat{f}_i - f_i)^2},$$

$$\mathrm{RelativeL2} = \frac{\|\hat{f} - f\|_2}{\|f\|_2},$$

$$E_{\mathrm{ODE}}^{\mathrm{RMS}} = \sqrt{\frac{1}{N}\sum_i \left|\hat{q}_i'' + 2\gamma\hat{q}_i' + (\gamma^2+\omega^2)\hat{q}_i\right|^2},$$

$$E_{\mathrm{hard},\infty} = \|A\beta - g\|_\infty,$$

以及

$$\kappa_2(A).$$

CNF 论文明确把 condition number 视为硬约束数值稳定性的关键量，官方训练代码也直接计算 `torch.linalg.cond(X)` 并记录为训练指标。 [1]

每个 `metrics.csv` 至少包含以下列：

```
run_id
seed
method
split
sample_id
A_true
gamma_true
omega_true
b_true
K
M
n_obs
collocation_type
noise_level
sigma
siren_width
siren_layers
omega0
solver
rmse
relative_l2
max_abs_error
ode_rms_dense
ode_max_dense
ode_rms_collocation
bc_left_abs
bc_right_abs
constraint_inf_norm
cond_A
log10_cond_A
solver_time_ms
train_time_s
inference_time_ms
num_parameters
numerical_failure
checkpoint_path
```

必须输出的图如下：

图	横轴 / 内容	纵轴 / 内容	用途
重建曲线	x	GT、prediction、observations	最直观比较四方法
点对点误差	x	$\lvert\hat{f} - f\rvert$	看衰减尾部是否恶化
ODE residual 曲线	x	$\lvert\mathcal{L}\hat{q}\rvert$，log scale	区分只在配置点满足与全域满足

图	横轴 / 内容	纵轴 / 内容	用途
boundary zoom	$x \approx 0, 1$	函数值	验证 CNF boundary 是否 machine-precision
condition vs K	K	median $\log_{10} \kappa(A)$	研究 basis capacity/stability
condition vs M	M	median $\log_{10} \kappa(A)$	研究约束密度
RMSE vs K	K	test RMSE	表达能力
residual vs K	K	dense ODE RMS	物理一致性
sigma sensitivity	σ	RMSE + cond	Gaussian bandwidth 选择
omega0 sensitivity	ω_0	RMSE + derivative error	SIREN 频率敏感性
coefficient heatmap	basis index × sample	β_j	分析基函数利用
A matrix heatmap	row × col	$\log \lvert A_{ij} \rvert$	观察 Gaussian/Fourier 结构
singular-value spectrum	singular-value index	$\sigma_i(A)$	解释病态来源
noise degradation	noise %	RMSE / ODE residual	鲁棒性
Pareto plot	train/solver time	Relative-L2	性能成本比较

论文结果表建议至少报告：

Method	K	M	Obs	Noise	RMSE ↓	Rel-L2 ↓	Dense ODE RMS ↓	Hard error ↓	median cond(A) ↓	Solver ms ↓	Train s ↓

对 CNF-SIREN 和 Gaussian 还应增加 derivative error：

$$E_{d1} = \|\hat{f}' - f'\|_2 / \|f'\|_2, \qquad E_{d2} = \|\hat{f}'' - f''\|_2 / \|f''\|_2,$$

因为这是 SIREN 与 CNF 高阶微分约束是否真正学得平滑函数的关键，而不是只看 function-value RMSE。SIREN 原论文专门强调了对信号导数的表示能力。[17]

环境、复现性与资源预算

推荐环境

为了减少兼容问题，建议维护两个独立环境。

环境	用途	推荐配置
`cnf_ref`	忠实参考官方 CNF	Python 3.9，PyTorch 1.13.0，Lightning 1.8.1，SciPy 1.9.3

环境	用途	推荐配置
`fundiff_ref`	仅用于核对原数据/原实验	当前 FunDiff JAX/Flax 环境
`cnf_modern`	后续工程重构，可选	Python 3.10/3.11 + 当前稳定 PyTorch；需要改写旧 functorch 接口

官方 CNF `requirements.txt` 明确锁定 PyTorch 1.13.0、Lightning 1.8.1、SciPy 1.9.3；官方 CNF README 将其标为官方实现。[18] 因此第一轮复现不建议直接升级所有依赖。SIREN 官方仓库虽然较旧，但我们只需要其网络和初始化原则，不需要完整旧环境。[19]

随机性必须统一记录：

```
random.seed(seed)
np.random.seed(seed)
torch.manual_seed(seed)
torch.cuda.manual_seed_all(seed)

torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False
```

每次 run 保存：

```
config.yaml
metrics.csv
summary.json
environment.txt
git_commit.txt
train.log
checkpoint_best.pt
checkpoint_last.pt
predictions.npz
figures/
```

`summary.json` 保存最终指标、失败原因、condition-number 极值、best step。`predictions.npz` 保存 dense x、GT、prediction、ODE residual、collocation indices 和 β，这样以后无需重训即可重新画图。

Checkpoint 策略建议：CNF 每 500 steps 保存一次，只保留 `best + last`；SIREN-PINN 每 1000 steps；Fourier 无训练 checkpoint，只保存频率集合、A、β 和配置。CNF inference 时 β 只需计算一次，论文明确指出其后预测只是 basis 的线性组合。[4]

资源估计

下面是**工程估算，不是论文报告值**；真实时间高度依赖 CPU、GPU、PyTorch 编译/自动微分开销以及是否对 100 个函数逐个训练。

场景	CNF-Gaussian / 单函数	CNF-SIREN / 单函数	Fourier / 单函数	SIREN-PINN / 单函数	峰值内存预估
CPU，MVP	10–60 s	30 s–3 min	<1 s	1–5 min	<1 GB
中端 GPU，MVP	3–20 s	5–40 s	CPU 即可	10–60 s	1–2 GB
CPU，100 functions	20 min–2 h	1–5 h	<1 min	2–8 h	通常 <2 GB
GPU，100 functions	5–30 min	10–60 min	<1 min	20–120 min	1–4 GB

这里的矩阵只有约 33×33 或 65×65，所以**线性 solve 本身不是主要计算瓶颈**；主要成本来自 neural basis 的高阶自动微分和反向传播。这与 CNF 论文讨论的规模性质一致：约束矩阵大小随 constraint 数量增长，而 basis derivative 和矩阵构建是重要成本；大规模问题才需要论文提出的 hybrid/sparse solver。 [4]

最终推荐的**第一条正式运行命令所对应的配置**应固定为：

```
experiment: cnf_gaussian_mvp
seed: 42

data:
  n_functions: 1
  n_sensors: 128
  n_obs: 32
  noise: 0.0
  save_parameters: true

physics:
  equation: damped_oscillator
  M: 31
  collocation: chebyshev
  boundary:
    type: dirichlet
    left_x: 0.0
    right_x: 1.0
    hard: true

basis:
  type: gaussian_kernel
  K: 33
  sigma: 0.05
  encoder: positional
  n_freqs: 3
  learn_encoder: true

solver:
  type: direct
  track_condition_number: true
```

```
  warn_cond: 1.0e6
  fail_cond: 1.0e12

training:
  optimizer: adam
  lr: 1.0e-3
  max_steps: 5000
  early_stop_patience: 500
  cond_weight: 1.0e-6

evaluation:
  dense_grid: 2048
  save_beta: true
  save_matrix: true
  save_residual_curve: true
```

这套 MVP 最先需要通过三个单元测试：解析目标函数在 dense grid 上的 ODE residual 应接近浮点误差；`A @ beta - g` 应接近线性求解精度；训练前后 `cond(A)`、dense ODE residual 和 reconstruction RMSE 必须都记录，而不能只看 loss。CNF 论文的硬约束机制、可微线性 solve、Gaussian kernel 推荐以及 condition-number regularization 都直接支持这一实验设计。 [1]

优先参考源为 CNF 的 NeurIPS 2023/arXiv v2 方法正文与官方 GitHub；CNF 官方仓库直接提供 basis、kernel、SIREN hypernetwork、高阶微分和求解实现。 [20] SIREN 的网络结构与初始化应参考 Sitzmann 等人的论文和官方仓库。 [12] FunDiff 的数据公式、参数范围和传感器设置应以其 `damp_sine/data_utils.py` 与配置文件为准。 [5]

---

[1] [4] [11] Neural Fields with Hard Constraints of Arbitrary Differential Order
https://rt.http3.lol/index.php?q=aHR0cHM6Ly9hcnhpdi5vcmcvaHRtbC8yMzA2LjA4OTQzdjI

[2] [5] fundiff/damp_sine/data_utils.py at main · sifanexisted/fundiff · GitHub
https://github.com/sifanexisted/fundiff/blob/main/damp_sine/data_utils.py

[3] [18] CNF/requirements.txt at main · zfc946/CNF · GitHub
https://github.com/zfc946/CNF/blob/main/requirements.txt

[6] fundiff/damp_sine/configs/autoencoder.py at main · sifanexisted/fundiff · GitHub
https://github.com/sifanexisted/fundiff/blob/main/damp_sine/configs/autoencoder.py

[7] [8] [9] [16] CNF/network.py at main · zfc946/CNF · GitHub
https://github.com/zfc946/CNF/blob/main/network.py

[10] [19] GitHub - vsitzmann/siren: Official implementation of "Implicit Neural Representations with Periodic Activation Functions" · GitHub
https://github.com/vsitzmann/siren

[12] [17] Implicit Neural Representations with Periodic Activation Functions
https://arxiv.org/abs/2006.09661?utm_source=chatgpt.com

[13] [15] CNF/nbf.py at main · zfc946/CNF · GitHub
https://github.com/zfc946/CNF/blob/main/nbf.py

[14] CNF/diff_utils.py at main · zfc946/CNF · GitHub
https://github.com/zfc946/CNF/blob/main/diff_utils.py

20 [2306.08943] Neural Fields with Hard Constraints of Arbitrary Differential Order
https://rt.http3.lol/index.php?q=aHR0cHM6Ly9hcnhpdi5vcmcvYWJzLzIzMDYuMDg5NDM